

Programmation procédurale : conditions et répétitives



On parle de quoi ?

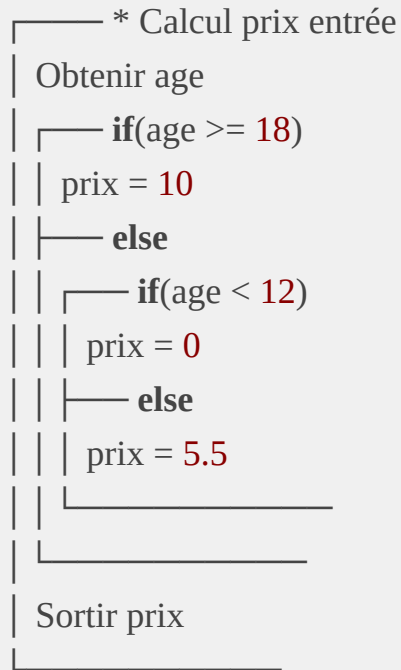
1. [Alternatives](#)
2. [Opérateurs](#)
3. [Simplifications](#)
4. [Alternative](#) [switch](#)
5. [Répétitives](#)
6. [Instruction](#) [break](#)
7. [Instruction](#) [continue](#)
8. [Le problème du tampon \(buffer\)](#)
9. [Exercice de synthèse](#)

Alternatives

Pourquoi utiliser des alternatives ?

Les alternatives permettent de faire réagir différemment un programme en fonction des entrées qui lui sont données.

Objectif : transcrire le diagramme suivant en C.



Pourquoi utiliser des alternatives ?

```
void main(void){  
    int age;  
    double prix;  
  
    printf("Entrez votre âge :");  
    scanf_s("%d", &age);  
  
    if(age >= 18){  
        prix = 10;  
    } else {  
        if(age < 12){  
            prix = 0;  
        } else {  
            prix = 5.5;  
        }  
    }  
  
    printf("Votre prix d'entrée est de %.2f euros : ", prix);  
}
```

Opérateurs

Opérateurs de comparaison

- plus grand : `>`
- plus petit : `<`
- plus grand ou égal : `>=`
- plus petit ou égal : `<=`
- égal : `==`
- différent : `!=`

→ quelle est la différence entre `a = 3` et `a == 3` ?

Opérateurs logiques

- ET : `&&`
- OU : `||`
- Négation : `!`

Exemple

```
if(nbVies == 0){
    estMort = true;
} else {
    estMort = false;
}

if(estMort && !peutRessusciter){
    // Ici, estMort = true et peutRessusciter = false
    printf("Perdu !");
} else {
    // Ici, estMort = false ou peutRessusciter = true (ou les deux)
    if(peutRessusciter){
        nbVies = 5;
    }
    printf("La partie continue ! Il vous reste %d vies", nbVies);
}
```

Les erreurs à éviter

- Utiliser `==` ou `!=` avec une variable booléenne

```
if(estMort == true && peutRessusciter != true){  
    ...  
}
```

- Confondre affectation et comparaison

```
if(nbVies = 0){  
    ...  
}
```

→ !! pas d'erreur lors de la compilation !!

Exercice

Créez un programme qui prend en entrée deux entiers (a et b) et qui affiche le message :

- "le produit est positif" si le produit de a et b est positif
- "le produit est négatif" si le produit de a et b est négatif
- "le produit est nul" si le produit de a et b est nul

Exercice

Créez un programme qui prend en entrée deux entiers (a et b) et qui affiche le message :

- "le produit est positif" si le produit de a et b est positif
- "le produit est négatif" si le produit de a et b est négatif
- "le produit est nul" si le produit de a et b est nul

```
if(a == 0 || b == 0){  
    printf("Le produit est nul")  
} else {  
    if((a < 0 && b < 0) || (a > 0 && b > 0)){  
        printf("Le produit est positif")  
    } else {  
        printf("Le produit est négatif")  
    }  
}
```

Simplifications

Dans certains cas, pour rendre le code plus concis (et donc facile à lire), on utilise l'alternative simplifiée.

```
// ? signifie "alors"
```

```
// : signifie "sinon"
```

```
bool max = (nbr1 <= nbr2) ? nbr2 : nbr1;
```

```
bool estPair = (mois % 2 == 0) ? true : false;
```

Switch

Le **switch** permet d'effectuer une série de test d'égalité sur une variable spécifique.

Par exemple, le code suivant :

```
char codeSexe = 'f';
if(codeSexe == 'f'){
    printf("Bonjour madame\n");
} else {
    if(codeSexe == 'h'){
        printf("Bonjour monsieur\n");
    } else {
        printf("Bonjour\n");
    }
}
```

Switch

Le **switch** permet d'effectuer une série de test d'égalité sur une variable spécifique.

Devient :

```
char codeSexe = 'f';
switch(codeSexe){
    case 'f' : printf("Bonjour madame\n");
        break;
    case 'h' : printf("Bonjour monsieur\n");
        break;
    default: printf("Bonjour\n");
}
}
```

Répétitives

Pourquoi utiliser des répétitives ?

Une répétitive permet de répéter l'exécution d'un bloc un certains nombre de fois en fonction d'une condition logique.

Objectif : traduire le diagramme d'action suivant.

```
┌── * Salutation
│  Obtenir nbSalutations
│  n = 0;
│
│  ┌── while(n < nbSalutations)
│  │  Sortir "Hey, salut toi !"
│  │  n++
│  └──
└──
```

Pourquoi utiliser des répétitives ?

```
void main (void){  
    int nbSalutations;  
    int n = 0;  
  
    printf("Combien de fois voulez-vous être salué ? ");  
    scanf_s("%d", &nbSalutations);  
  
    while(n < nbSalutations){  
        printf("Hey, salut toi !\n");  
        n++;  
    }  
}
```

Les différentes répétitives

- l'instruction **while**

```
int nombre = -10;  
while(nombre < 0){  
    nombre += 2;  
}
```

- l'instruction **do ... while**

```
int nombre = -10;  
do{  
    nombre += 2;  
} while (nombre < 0)
```

- l'instruction **for**

```
for(int cpt = 0; cpt < 10; cpt++){  
    printf("Je suis déjà passé %d fois ici\n", cpt + 1);  
}
```

L'instruction **break**

Objectif : arrêter un processus répétitif.

```
char reponse;
while(1){
    printf("Souhaitez-vous arrêter la boucle ? (o/n)\n");
    scanf_s("%c", &reponse, 1);
    viderTampon();
    if(reponse == 'o') break;
}
```


L'instruction **continue**

Objectif : ignorer une partie d'un processus répétitif.

```
char reponse;
while(1){
    printf("Souhaitez-vous aller plus loin ? (o/n)\n");
    scanf_s("%c", &reponse, 1);
    viderTampon();
    if(reponse == 'n') continue;
    printf("Vous avez été plus loin !\n");
}
```

Exercices de synthèse

Créez un programme qui calcule le montant d'une amende d'un excès de vitesse :

- en agglomération, si la vitesse était entre 51km/h et 60km/h, l'amende s'élève à 53€ ; ensuite, chaque km/h supplémentaire coûte 11€
- hors agglomération, l'amende est de 53€ les 10km/h qui suivent la limite autorisée ; ensuite, chaque km/h supplémentaire coûte 6€

Votre programme demandera à l'utilisateur :

- la vitesse à laquelle il roulait
- le lieu de l'infraction (agglomération ou hors agglomération)
- si nécessaire, la vitesse limite autorisée sur le lieu de l'infraction

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
void main(void){
```

```
    int vitesse;
```

```
    bool enAgglo;
```

```
    char reponse;
```

```
    int montant;
```

```
    int limite;
```

```
    printf("Entrez votre vitesse au moment de l'infraction :\n");
```

```
    scanf_s("%d", &vitesse);
```

```
    viderTampon();
```

```
    printf("Etiez-vous en agglomération ? (o/n)\n");
```

```
    scanf_s("%c", &reponse, 1);
```

```
    viderTampon();
```

```
    enAgglo = (reponse == 'o') ? true : false;
```

```
if(enAgglo){
    if(vitesse < 51){
        montant = 0;
    } else {
        montant = 53;
        if(vitesse > 60){
            montant += (vitesse - 60) * 11;
        }
    }
} else {
    printf("Entrez la vitesse limite sur le lieu de l'infraction :\n");
    scanf_s("%d", &limite);

    if(vitesse <= limite){
        montant = 0;
    } else {
        montant = 53;
        if(vitesse > limite + 10){
            montant += (vitesse - limite - 10) * 6;
        }
    }
}
printf("Le montant de l'amende s'élève à %d€\n", montant);
}
```

Exercices de synthèse

Créez un programme qui demande un entier à l'utilisateur et qui affiche un triangle (plein) en forme d'étoiles ayant une hauteur de taille équivalente à cet entier.

Exemple : si l'utilisateur entre la hauteur 5, le triangle aura la forme suivante.

```
*  
***  
*****  
*****  
*****
```

```
void main (void){  
    int hauteur;  
    int nbEtoiles = 1;  
  
    printf("Entrez la hauteur du triangle :\n");  
    scanf_s("%d", &hauteur);  
  
    for(int nEtage = 0; nEtage < hauteur; nEtage++){  
        for(int nEspace = hauteur - nEtage; nEspace > 0; nEspace--) printf(" ");  
        for(int nEtoile = 0; nEtoile < nbEtoiles; nEtoile++) printf("*");  
        nbEtoiles += 2;  
        printf("\n");  
    }  
}
```